DOCUMENTAZIONE DEL PROGETTO

# Redstone Studio

Simulatore di Circuiti Redstone

| | |
|---|---|
| **Classe** | 4J |
| **Anno Scolastico** | A.S. 2025/2026 |
| **Tecnologia** | C# · .NET 8 · Avalonia UI · MVVM |


COMPONENTI DEL GRUPPO

- Dmytriy Kovalyshyn
- Filippo Tognoni
- Leonida Biondi
- Luca Pischettola

# Indice


1 Introduzione

2 Analisi

3 Progettazione della GUI

4 Problemi Riscontrati

5 Implementazione

# 1 Introduzione

Il progetto **Redstone Studio** è un simulatore semplificato dei circuiti redstone di Minecraft. La redstone funziona come un filo elettrico: trasmette segnali tra componenti che si comportano da interruttori, amplificatori, invertitori e indicatori luminosi.

L'applicazione permette di posizionare componenti su una griglia, avviare la simulazione tick per tick e annullare o ripetere le modifiche alla griglia.

Il progetto è sviluppato in **C#** con **.NET 8**, usando **Avalonia UI** per l'interfaccia grafica e il pattern **MVVM**. La struttura segue la *Clean Architecture*, con tre layer separati: dominio, applicazione e presentazione.

# 2 Analisi

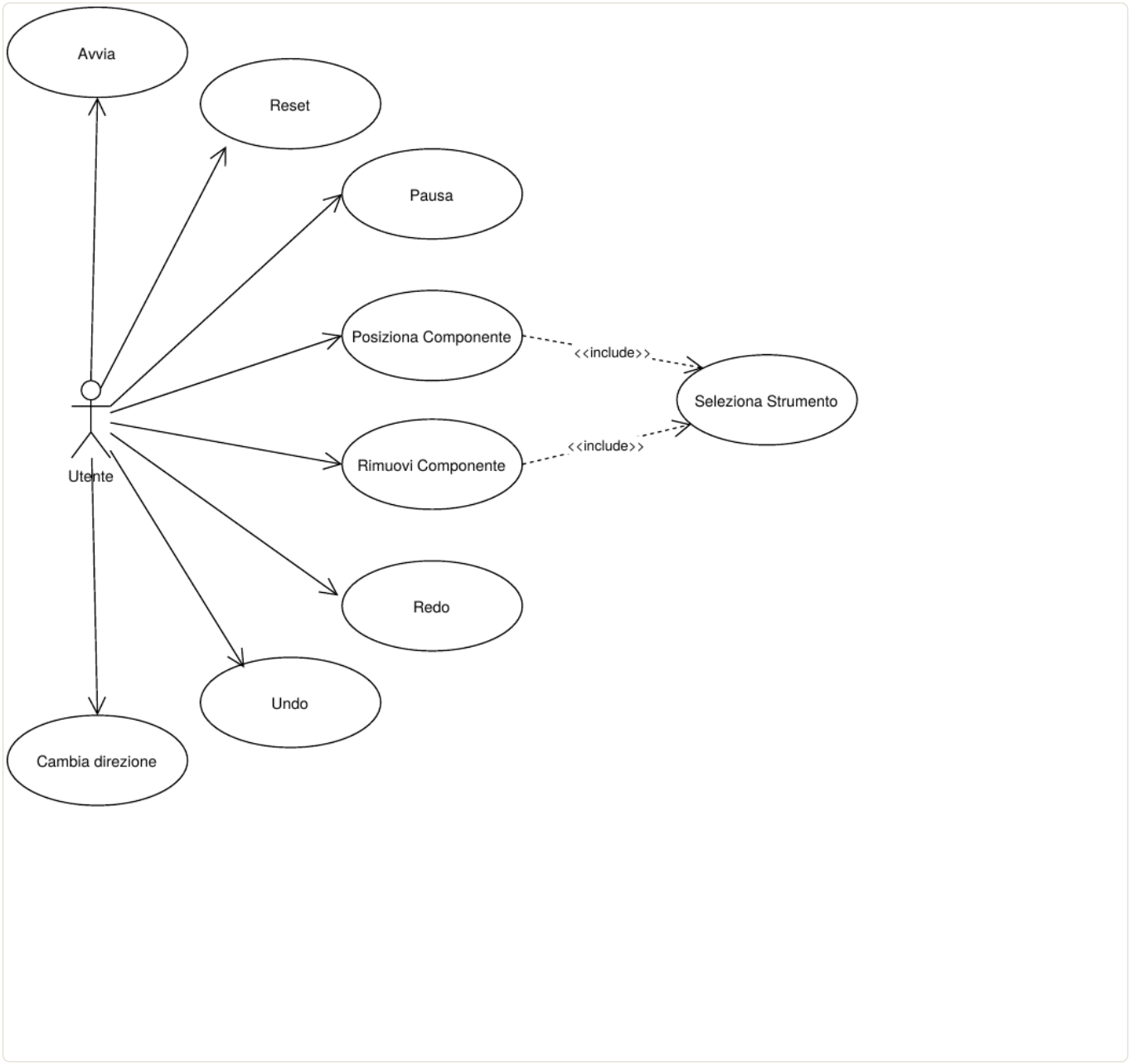
## Requisiti Funzionali

| Requisito |
|---|
| L'utente può selezionare un componente dalla toolbar e posizionarlo sulla griglia. |
| Cliccando sulla stessa cella con lo stesso strumento, il componente cambia stato (es. accendere la leva). |
| Lo strumento "Cancella" rimuove il componente dalla cella. |
| I componenti orientabili (torcia, ripetitore) si posizionano nelle quattro direzioni. |
| Il motore di simulazione propaga correttamente segnale, inversione della torcia e ritardo del ripetitore. |
| L'utente può avviare, mettere in pausa, avanzare di un tick e azzerare la simulazione. |
| L'utente può annullare (Undo) e ripetere (Redo) le modifiche alla griglia. |

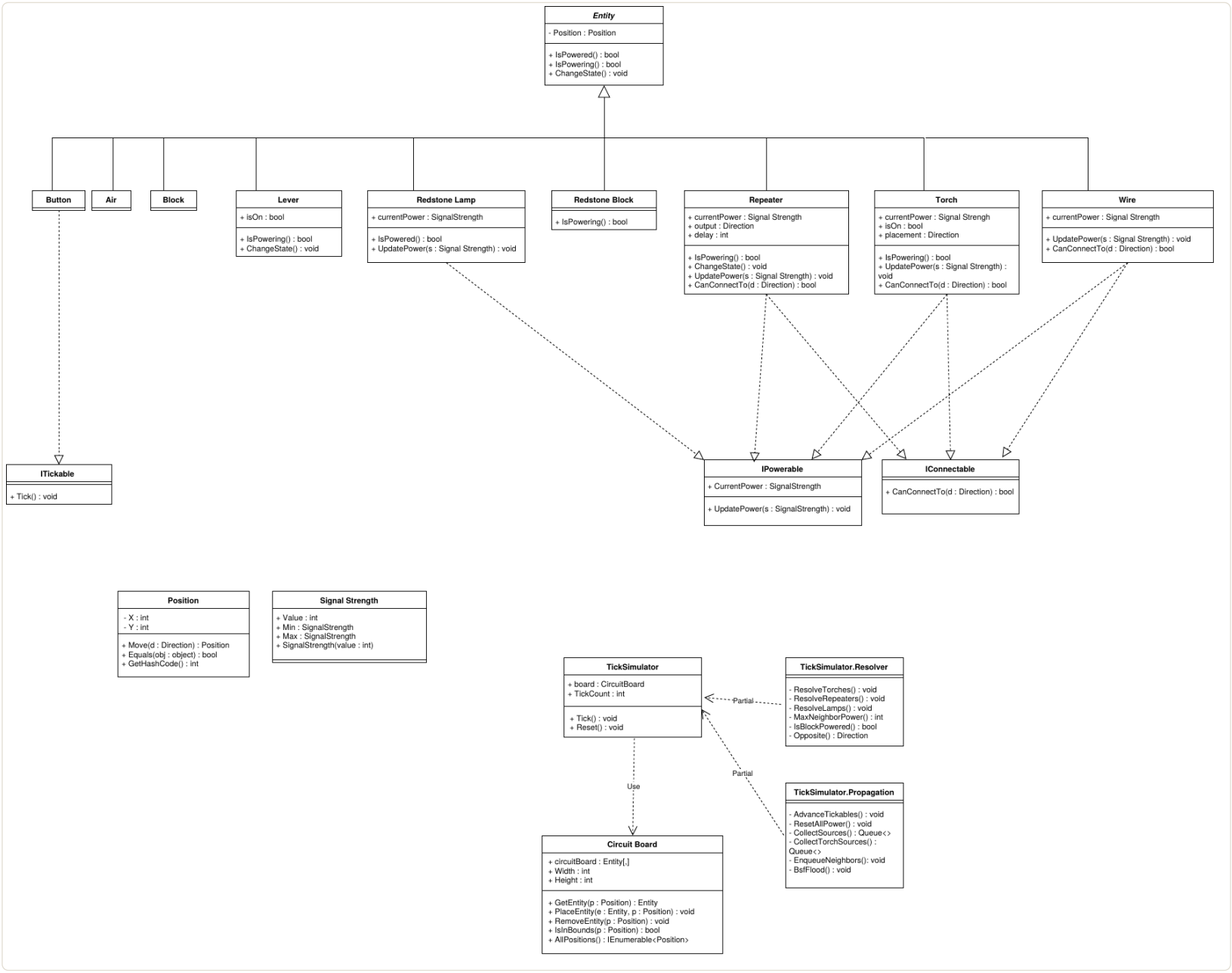
## Componenti Implementati

| Componente | Simbolo | Descrizione |
|---|---|---|
| **Wire** | · / 1–F | Filo redstone. Trasmette il segnale con decadimento di 1 per blocco (0–15). Mostra il livello di potenza come cifra. |
| **Lever** | ╱ / \| | Leva on/off. Sorgente permanente finché accesa. |
| **Button** | ○ / ● | Pulsante. Emette un impulso di 3 tick, poi si spegne. |
| **RedstoneBlock** | ■ | Sorgente costante a potenza 15. |
| **Torch** | ↑ / ↓ | Torcia redstone. Invertitore: accesa se il suo ingresso non è alimentato. |
| **Repeater** | ▲ ▶ ▼ ◀ | Amplifica il segnale a 15 con ritardo da 1 a 4 tick. |
| **RedstoneLamp** | ✧ / ✦ | Lampada. Si illumina quando riceve segnale. Non trasmette oltre. |
| **Block** | ▉ | Blocco solido non conduttivo. Può essere attraversato dal segnale di un ripetitore. |

## Use Case Diagram (UCD)



# Class Diagram (CD)



# 3 Progettazione della GUI

L'interfaccia usa un tema scuro ispirato a Minecraft. La finestra è strutturata con un `DockPanel`: in cima una toolbar con gli strumenti, i controlli di direzione e i pulsanti Undo/Redo, al centro la griglia 40×20 in uno `ScrollViewer`, in fondo i pulsanti di simulazione (Play, Pause, Step, Reset) e il contatore dei tick.

La griglia usa un `ItemsControl` con `UniformGrid` a 40 colonne. Ogni cella è un `Button` di 36×36 pixel con sfondo colorato e simbolo Unicode centrato. Per evitare spazi vuoti tra le celle quando la finestra è grande, l'`ItemsControl` è impostato con `HorizontalAlignment="Left"` e `VerticalAlignment="Top"`, così occupa solo lo spazio necessario.

# 4 Problemi Riscontrati

### BFS Non Propagava il Segnale

Le sorgenti (Lever, RedstoneBlock, Button) venivano inserite nel BFS come punti di partenza. Il BFS però elabora solo celle Wire: trovando una Lever, la saltava immediatamente senza propagare nulla. Il circuito rimaneva sempre spento.

**SOLUZIONE** `CollectSources()` non accoda la posizione della sorgente, ma le sue quattro celle adiacenti a potenza 15. Così il BFS parte già dai fili vicini.

### Lampada Non Si Illuminava all'Ultimo Salto

Un Wire a potenza 1 propagava al vicino con `power - 1 = 0`, quindi la lampada adiacente non riceveva mai segnale anche se era fisicamente connessa.

**SOLUZIONE** Le lampade sono escluse dal BFS. Il metodo `ResolveLamps()` le aggiorna per ultimo, leggendo direttamente la potenza dei vicini senza decrementarla.

### Ripetitore Non Propagava Oltre la Prima Cella

Quando il ritardo di un ripetitore scadeva, il codice aggiornava direttamente solo il Wire subito davanti. I fili successivi restavano a 0.

**SOLUZIONE** Quando un ripetitore scatta, invece di aggiornare direttamente un Wire, si aggiunge la posizione di uscita come seed BFS. Il metodo `BfsFlood()` propaga poi il segnale attraverso l'intera catena.

### Ripetitori in Serie Non Funzionavano

Con due ripetitori in serie (A → filo → B), il ripetitore B non veniva mai attivato. La lettura dell'input di B avveniva prima che il BFS dall'output di A girasse, quindi il filo tra A e B risultava ancora a 0.

**SOLUZIONE** All'interno di `ResolveRepeaters()`, il BFS dai ripetitori scattati viene eseguito prima di leggere gli input dei ripetitori successivi.

### Ripetitore Non Passava il Segnale Attraverso un Blocco

In Minecraft un ripetitore può "alimentare fortemente" un blocco solido: un altro ripetitore sul lato opposto di quel blocco riceve il segnale. Nel simulatore, il BFS si ferma ai Wire e non entra nei Block, quindi il secondo ripetitore non vedeva nessun segnale.

**SOLUZIONE** In `ResolveRepeaters()`, se il segnale di un ripetitore arriva su un Block, quel blocco viene aggiunto a un insieme `_poweredBlocks`. Quando si legge l'input di un altro ripetitore, se la cella davanti è in `_poweredBlocks`, il valore viene letto come 15.

### Lampade che Passavano il Segnale

Le lampade implementano l'interfaccia `IPowerable`, quindi `MaxNeighborPower()` le contava come sorgenti di potenza. Una lampada illuminata trasmetteva il segnale alle lampade adiacenti come se fosse un filo.

**SOLUZIONE** Aggiunto `if (e is RedstoneLamp) continue` in `MaxNeighborPower()` per escluderle dal calcolo.

# 5 Implementazione

## Struttura del Progetto

Il codice è diviso in tre layer:

```
redstoneStudio.UI          – interfaccia grafica (Avalonia, MVVM)
RedstoneStudio.Application – motore di simulazione (TickSimulator, CircuitBoard, BoardSnapshot)
RedstoneStudio.Domain      – entità, interfacce, value objects
```

Ogni layer dipende solo da quelli più interni. Il dominio non conosce nulla degli altri layer.

## Il Ciclo di Tick

`Tick()` è il metodo principale del simulatore. Viene chiamato ogni 200 ms in modalità Play oppure una volta quando si preme Step.

```
public void Tick()
{
    TickCount++;
    AdvanceTickables();              // 1. countdown del pulsante
    ResetAllPower();                 // 2. azzera tutto a 0
    BfsFlood(CollectSources());      // 3. propaga da lever / button / redstone block
    ResolveTorches();                // 4. aggiorna le torce
    BfsFlood(CollectTorchSources()); // 5. propaga dalle torce accese
    ResolveRepeaters();              // 6. gestisce i ritardi dei ripetitori
    ResolveLamps();                  // 7. accende le lampade
}
```

L'ordine è importante: le torce devono essere aggiornate **dopo** il primo BFS, altrimenti leggerebbero i fili ancora a 0.

## BFS e Propagazione del Segnale

`BfsFlood()` propaga la potenza attraverso le celle Wire usando una coda (Breadth-First Search). L'algoritmo parte da una serie di celle "seme" con potenza iniziale 15 e si espande verso i vicini, diminuendo la potenza di 1 ad ogni salto. Ogni cella viene aggiornata solo se la nuova potenza è strettamente maggiore di quella già presente: questo garantisce che il segnale non torni mai indietro con un valore più basso, evitando cicli infiniti senza bisogno di tenere traccia delle celle già visitate.

Le celle non-Wire (torce, ripetitori, lampade, blocchi solidi) fermano la propagazione: il BFS le dequeue e le salta senza fare nulla. Per questo motivo le sorgenti di segnale (Lever, RedstoneBlock, Button) non vengono inserite direttamente nella coda, ma vengono invece accodate le loro quattro celle adiacenti a potenza 15: così il BFS inizia già dai fili vicini.

```
private void BfsFlood(Queue<(Position pos, int power)> seeds)
{
    while (seeds.Count > 0)
    {
        var (current, power) = seeds.Dequeue();

        if (!_board.IsInBounds(current) || power <= 0) continue;

        var entity = _board.GetEntity(current);

        if (entity is not Wire wire) continue;           // propaga solo tra Wire
        if (power <= wire.CurrentPower.Value) continue;  // non sovrascrivere valori più alti

        wire.UpdatePower(new SignalStrength(power));

        // Accoda i 4 vicini con potenza diminuita di 1
        foreach (Direction dir in Enum.GetValues<Direction>())
        {
            var next = current.Move(dir);
            if (_board.IsInBounds(next))
            {
                seeds.Enqueue((next, power - 1));
            }
        }
    }
}
```

Il BFS viene chiamato due volte per tick: la prima volta dalle sorgenti dirette, la seconda volta dalle torce accese. Questo ordine è necessario perché le torce devono prima leggere la potenza dei fili vicini (prodotta dal primo BFS) per decidere se accendersi o spegnersi.

## Torce e Ripetitori

La torcia usa la logica inversa: `_on = (incomingPower.Value == 0)`. Si accende quando non riceve segnale, si spegne quando lo riceve.

Il ripetitore usa una coda `_pendingRepeaters` che sopravvive tra un tick e l'altro. Ogni voce contiene la posizione, la direzione di uscita e i tick rimanenti. Quando il contatore arriva a 0, il ripetitore scatta e viene avviato un BFS dal punto di uscita. Un blocco solido davanti al ripetitore viene "alimentato fortemente", permettendo al segnale di attraversarlo verso un altro componente.

## Entità di Dominio

Tutte le entità ereditano da `Entity` e implementano solo le interfacce pertinenti:

- **IPowerable** (Wire, Torch, Repeater, RedstoneLamp): `CurrentPower` e `UpdatePower()`
- **IConnectable** (Wire, Torch, Repeater): `CanConnectTo(Direction)`
- **ITickable** (Button): `Tick()` per il countdown

`Lever` e `RedstoneBlock` sono sorgenti dirette e non implementano `IPowerable`.

## Undo / Redo

Il `MainWindowViewModel` mantiene due stack di `BoardSnapshot`: `_undoStack` per annullare e `_redoStack` per ripetere. `BoardSnapshot` è una classe nel layer Application che cattura lo stato della griglia come lista di `CellSnapshot` (tipo, posizione, parametri specifici di ogni entità). Prima di ogni modifica alla griglia, lo stato corrente viene salvato in `_undoStack` e `_redoStack` viene svuotato. Lo stack è limitato a 50 voci.

I pulsanti "Undo" e "Redo" nella toolbar si abilitano e disabilitano automaticamente in base alla presenza di voci nei rispettivi stack.

```
private void OnCellClicked(CellViewModel cell)
{
    // Snapshot dello stato corrente prima di ogni modifica
    _undoStack.Add(BoardSnapshot.Capture(_board));
    _redoStack.Clear();
    if (_undoStack.Count > 50)
    {
        _undoStack.RemoveAt(0);
    }
    // applica la modifica...
}
```

## Interfaccia Grafica

Il `MainWindowViewModel` collega la simulazione alla UI. Gestisce la `ObservableCollection<CellViewModel>` che rappresenta la griglia. Cliccando una cella si verifica uno di tre casi: rimuovere (strumento Cancella), cambiare stato (stessa entità già presente) o posizionare un nuovo componente.

Ogni `CellViewModel` riceve nel costruttore un `Action<CellViewModel>` che punta al metodo `OnCellClicked` del ViewModel principale. Questo sostituisce i binding dinamici con un semplice delegato, evitando problemi con i compiled bindings di Avalonia.

I fili mostrano il livello di potenza come carattere: `·` per potenza 0, `1`–`9` per potenza 1–9, `A`–`F` per potenza 10–15.